02 — Engineering & Operations Lessons

Cập nhật 2026-10-29 (branch `hmm-navigation`). Tài liệu này chỉ về kỹ thuật và vận hành: code, môi trường, Kaggle, cách chạy thí nghiệm, lỗi đã gặp và cách sửa, bài học. Viết trong giai đoạn SelfCal; hạ tầng Kaggle và các điểm dễ sai của loader vẫn đúng cho hướng HMM. Các script thí nghiệm SelfCal được nhắc tới (`eval_all.py`, `exp*`) nay chỉ còn ở branch `claude/selfcal-aa` / `claude/selfcal-seq`. Bảng tài khoản và dataset: `ENVIRONMENT_SUMMARY.md`.

1. Chính sách tính toán (user chỉ định)

1. **Mọi tính toán nặng chạy trên Kaggle**, kể cả phân tích chỉ cần CPU. Container local (không GPU, 4 nhân) chỉ dùng cho kiểm tra nhanh, dưới khoảng 2 phút.
2. **Dùng hết GPU T4 của mọi acc trước** (8 acc × 2 = 16 slot). Chỉ dùng kernel CPU khi hết GPU **và** job đủ nhẹ.
3. Không ghi token vào `~/.bashrc` (bị từ chối). Token để ở `~/.kaggle/tok_<username>`; script tự đọc.
4. Không chạy code bên thứ ba tải về ngay trong container (sandbox chặn). Chạy nó trong kernel Kaggle.

2. Pipeline đánh giá trên Kaggle (đang dùng, đã kiểm chứng)

```
python3 automation/fanout.py <work_dir> <prefix> "eval_all.py rc salad;eval_all.py msls megaloc;..." [n_kerne
```

- `fanout.py` chia job theo vòng vào n kernel, rải trên (acc, slot): slot 0 của mọi acc trước, rồi đến slot 1. Nó chờ dataset code ở trạng thái `ready`, chạy `run_kernel.py` cho từng kernel, chờ xong rồi copy `results/*.json` vào `analysis/results/`.
- `make_eval_kernel.py <dir> <slug> "<jobs>" "" [--gpu] [--owner <user>]`:
- upload `analysis/*.py` thành dataset code `<owner>/dive-vpr-analysis` (mỗi lần tạo một version mới);
- viết `run.py`: mount dataset dump public `nhatlong1103/vpr-descriptor-dumps`, symlink vào `dump_all/`, chạy job song song (cpu − 1 luồng), lưu `results/`, `logs/`, `figs/`.
- Job có thể chứa token chữ `DUMP` ở chỗ cần đường dẫn dump (ví dụ `exp10_shiftfix.py salad DUMP`).
- `run_kernel.py <dir> <out> 600 3 14400`: push, poll trạng thái, tải output. Kernel bị QUEUED quá 10 phút thì `delete` rồi push lại. `--attach` để bám lại vào kernel đang chạy sau khi container khởi động lại.
- **Backend GPU:** `selfcal.qm_stats` dùng PyTorch khi có CUDA (`SELFCAL_DEVICE=auto|cuda|torchcpu|numpy`).
- Đã đối chiếu với numpy: RobotCar trùng hoàn toàn, Nordland lệch ≤ 0.00025.
- Trọng số SUE phải để float32 như numpy; trước đó float64 làm lệch 3 vi phạm ở α = 0.01.
- **Thời gian trên T4:**
- `eval_all` RobotCar/CMU/Nordland: 0.5–1 phút mỗi setting. MSLS: 27–36 phút.
- exp9 (CPU): 17–40 phút mỗi descriptor. exp10 mỗi descriptor: 11–17 phút.
- exp11/exp12: dưới 1 phút mỗi job.
- **Cache thống kê:** exp10 lưu `shiftstats_<desc>.npz` gồm mọi thống kê raw và centred. `exp10_shiftfix.py <desc> --stats <npz>` chấm lại biến thể mới trong vài giây (93 s → 2.5 s). Nên dùng cách này cho mọi tìm kiếm biến thể.

3. Kaggle: bài học vận hành

- **Ngắt một session:** chỉ `kaggle kernels delete -y <owner>/<slug>` có tác dụng. Push version mới hay push bản CPU "no-op" không ngắt session cũ. Cửa sổ interactive trên web phải tắt trên giao diện.
- **Trần concurrency:** 2 session GPU mỗi acc. Push lại mà không `delete` sẽ ăn vào trần và gây lỗi **Maximum batch GPU session count of 2**.
- **Log trễ:** API `output`/`logs` chỉ lộ log gần lúc xong. Kernel RUNNING mà chưa có output không có nghĩa là treo. Runner vì thế tắt chế độ silent-restart.
- **T4 và RTX PRO 6000:** T4 cấp ngay và chạy song song được. RTX có lần xếp hàng 21 phút với 0 output. Chỉ dùng RTX khi khối lượng extraction rất lớn.

- **Dataset code versioning:** sau `make_eval_kernel` phải chờ `kaggle datasets status` báo `ready` rồi mới push kernel (`fanout.py` làm sẵn). Không chờ thì kernel có thể mount version code cũ.
- **Giới hạn metadata:** title dataset dài 6–50 ký tự, subtitle 20–80. Title dài quá thì upload hỏng sau khi đã đẩy hết 4.4 GB.
- **Chia sẻ dữ liệu giữa các acc:** output kernel riêng tư của acc khác không mount được. Giải pháp hiện tại là dataset **public** `nhatlong1103/vpr-descriptor-dumps`; acc mới không cần upload gì. Hai bản dump riêng tư cũ trên acc 2 và acc 3 đã xoá.
- **Upload:** khoảng 18 MB/s (4.4 GB mất khoảng 6 phút với `-r zip -q`). Dùng hard link (`cp -al`) để chuẩn bị thư mục mà không tốn thêm dung lượng.
- **Acc mới:** đã thử GPU T4 trên cả 8 acc. Acc chưa xác minh số điện thoại sẽ không được cấp GPU, nên cần thử một job nhỏ trước.

4. Code: cấu trúc và quy ước

- `analysis/vprlib.py`: loader (`LOADERS[rc|cmu|nord|msls|nordw]`), metric, `nearest_map_dist`, `correct`, `top1_errors`. CMU gộp 11 slice. MSLS gộp hai thành phố, SF dịch 1e7 m theo UTM.
- `analysis/selfcal.py`:
- `query_stats`/`qm_stats` (chunk theo hàng, `reduceat` lấy max theo ô);
- `lobo_stats`, `map_null(stride=2)`, `conformal_p`, `combined_p`;
- `SelfCalTest(N, strata=10, gate=0.8, min_per=50, mode="aa"|"gap")`, `pack`, `selfcal_p`;
- lớp online `SelfCalAA` (khoảng 1 ms/query theo lô, 18 s chuẩn bị bản đồ RobotCar).
- `analysis/sc_eval.py`: `CFG` cho từng dataset (cell 10 m, excl 25 m, k 10, block 300 m, off_d 40 m, in_d, tol).
- Bảng paper: `paper/build_tables.py` (đọc `analysis/results/` và gộp thêm `results/gap/`) → `paper/tables.md`. `paper/build_paper.py` → `SelfCal_paper_full.pdf`.
- md → PDF: `python3 automation/md2pdf.py <in.md> <out.pdf>` (thuần Python, xhtml2pdf). Cần dòng trống trước mỗi danh sách. Ô bảng không ngắt dòng được, nên tránh chuỗi code dài trong ô.

5. Dữ liệu và loader: các điểm dễ sai

- **RobotCar `robotcar_v2_train.txt`:** pose 4×4 phẳng, translation nằm ở chỉ số [3, 7, 11]. Ảnh tham chiếu `overcast-reference` không có trong file train; pose của nó lấy từ `3D-models/all.nvm`.
- **CMU:** thư mục lồng `sliceN/sliceN/`; glob `**/slice*` bắt trùng nên phải dedup. Pose xyz là 3 số cuối. Chỉ dùng camera c0.
- **Nordland:** 4 mùa căn khung 1:1. Lấy mẫu con 4,500 trên 35,768 frame. Vị trí tính bằng frame.
- **MSLS:** CSV khi tải từ Kaggle nằm trong thư mục lồng nên phải di chuyển. pandas 3 trả StringArray, phải `np.asarray(..., dtype=str)` trước `np.savez`. Bỏ ảnh panorama.
- **Thứ tự frame cho sequence** (`vprlib.frame_order`, chỉ có trên branch seq): timestamp nằm trong tên ảnh. Bước giữa các frame liên tiếp khoảng 1 m, giống nhau giữa query và map.

6. Lỗi đã gặp và cách sửa (theo nhóm)

Lỗi	Cách sửa / bài học
`pkill -f <pattern>` giết luôn shell đang gọi (exit 144)	Dùng PID cụ thể; hoặc `TaskStop` cho tác vụ nền
Chuỗi chờ nền tự push lại kernel sau khi metadata đã đổi acc, suýt tạo bản trùng	Dừng chuỗi cũ (TaskStop) trước khi chuyển job sang acc khác
`sed` xoá nhầm dòng	Sửa file bằng Python replace có `assert` hoặc dùng Edit; kiểm tra lại ngay
`exp9` dùng `sc.n` sau khi đổi API	Đổi API thì grep mọi chỗ gọi (`sc.test.n`)
Định dạng chuỗi với giá trị `None` khi chỉ có 1 bản đồ	Dùng `nan` thay `None`
Bộ chạy đặt đường dẫn dump sai vị trí đối số	Thêm token chữ `DUMP`
Cùng một biến thể chạy lại nhiều lần, tốn Kaggle	Lưu cache thống kê trung gian (exp10 `--stats`)
Kernel CPU chạy trong khi GPU acc khác rảnh (user nhắc)	Quy tắc: dùng hết GPU mọi acc trước
Kết quả seed 0–2 lạc quan hơn 10 seed	Luôn dùng đủ seed (≥ 10) trước khi kết luận về demo

7. Bài học thành công và thất bại

- **Thành công:**
  - Hiệu chuẩn bằng chính bản đồ, cache thống kê, chạy fan-out trên nhiều acc, dataset dump public.
  - Đối chiếu từng bit sau mỗi thay đổi code (GPU với numpy, AA mặc định với code cũ).
  - Kiểm tra local nhỏ trước khi push.
- **Thất bại hoặc cần tránh:**
  - Chọn biến thể sau khi xem dữ liệu mà không kiểm định độc lập (gate 0.8, B = 10, Gap): phải nói rõ và kiểm định trên dữ liệu mới.
  - Mở rộng sang sequence mà không kiểm tra exchangeability của cửa sổ theo từng điều kiện.
  - Kết luận từ 3 seed.
- **Quy trình git:**
  - Branch ổn định `claude/selfcal-aa` không ghi đè. Thí nghiệm mở rộng làm trên branch riêng (`claude/selfcal-seq`).
  - Kết quả mới đặt ở thư mục riêng (`results/gap/`) thay vì sửa file cũ.
  - Không tạo PR. Commit kèm trailer, không ghi model ID.